A Mini Project Report

on

Multilingual Document Summarizer

Submitted to

AISSMS College of Engineering, Pune

In partial fulfillment of the requirements for the Third Year of the Degree of

BE/BTech

in

Artificial Intelligence and Machine Learning

Submitted by

1. Hardik Pathak – _____
2. _____
3. _____

Under the guidance of

Prof. _____

Department of Artificial Intelligence and Machine Learning

AISSMS College of Engineering

Kennedy Road, Pune – 411001, Maharashtra, India

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project report entitled “Multilingual Document Summarizer” is a record of bonafide work carried out by the students under the supervision and guidance of Prof. _____, in partial fulfillment of the requirements for the Third Year (Artificial Intelligence and Machine Learning) programme of AISSMS College of Engineering, Pune, during the academic year 2026–2027.

Date:

Place: Pune

Submitted by:

1. Hardik Pathak – _____
2. _____
3. _____

_____	_____
Prof. _____	Dr. _____
Project Guide	Head of Department

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide for providing valuable guidance, encouragement, and support throughout the development of our mini project, “Multilingual Document Summarizer”. Their valuable suggestions, constructive feedback, and continuous motivation helped us understand the project requirements and complete the work successfully.

We are thankful to our Head of Department and all faculty members for giving us the opportunity to work on this project as part of our Natural Language Processing (NLP) curriculum. Their guidance and support encouraged us to explore the practical applications of NLP and understand how modern sequence-to-sequence models can be used to solve real-world problems.

We would also like to extend our sincere thanks to our college for providing the necessary facilities, resources, and a positive learning environment throughout the project development process.

Through this project, we gained practical experience in text preprocessing, abstractive summarization using Transformer-based models (BART and IndicBART), model fine-tuning on Kaggle GPUs, ROUGE-based evaluation, and web application development using Python, FastAPI, and Streamlit.

We would also like to acknowledge the learning resources, technical documentation, and open-source datasets and models (CNN/DailyMail, XL-Sum, Hugging Face Transformers) that helped us understand the tools and technologies used in this project.

Finally, we are thankful to everyone who directly or indirectly supported us during the completion of this mini project. This experience helped us understand the importance of making information accessible across languages, and we hope that the system can contribute towards easier access to long documents for readers of English and Hindi.

ABSTRACT

Multilingual Document Summarizer is a web-based application that produces concise, abstractive summaries of long documents in English and Hindi. Large amounts of useful information are locked inside PDF files, Word documents, and plain-text files, and readers often need to spend considerable time locating the key points. Language is an additional barrier, because most existing summarization tools support English only.

The proposed system accepts .txt, .pdf, and .docx files, extracts their text, and passes it to a language-specific sequence-to-sequence model that generates an abstractive summary. English documents are summarized using BART fine-tuned on the CNN/DailyMail dataset, while Hindi documents are summarized using IndicBART fine-tuned on the XL-Sum dataset. Users can also paste text directly for summarization.

The application is developed using Python and the Hugging Face Transformers library, with text extraction handled by PyMuPDF and python-docx. The user interface is provided in two forms: a FastAPI-backed web application with an HTML/CSS/JavaScript frontend, and a Streamlit demo. The models were fine-tuned on a NVIDIA T4 GPU using the Hugging Face Trainer and evaluated using the ROUGE metric. An optional Gemini API backend allows the app to run without downloading local models.

Keywords: Natural Language Processing (NLP), Abstractive Summarization, BART, IndicBART, XL-Sum, Multilingual, Fine-tuning, ROUGE, FastAPI, Streamlit.

CONTENT

Chapter / Contents	Page No.
Acknowledgement	i
Abstract	ii
Table of Contents	iii
List of Figures	iv
List of Tables	v
Chapter 1: Introduction	1
1.1 Introduction to the Project	1
1.2 Background and Motivation	1
1.3 Problem Statement	2
1.4 Objectives	2
1.5 Scope of the Project	2
1.6 Applications of the Project	3
Chapter 2: Literature Review	4
2.1 Review of Existing Research Papers	4
2.2 Existing System	5
2.3 Limitations of Existing System	5
2.4 Research Gap	5
Chapter 3: Problem Statement and Requirements	6
3.1 Problem Definition	6
3.2 Proposed Solution	6
3.3 Functional Requirements	6
3.4 Non-Functional Requirements	7
3.5 Hardware and Software Requirements	7
Chapter 4: Proposed Methodology	8
4.1 Overview of the Proposed System	8
4.2 System Architecture	8
4.3 Workflow of the System	9
4.4 NLP Techniques and Algorithms	9
4.5 Dataset Collection and Preprocessing	10
Chapter 5: System Design and Implementation	11
5.1 System Design	11
5.2 Flowchart / UML / DFD Diagrams	11
5.3 Tools, Technologies and Libraries	12
5.4 Implementation Details	12

5.5 User Interface and Screenshots	13
Chapter 6: Results and Discussion	14
6.1 Experimental Setup	14
6.2 Test Cases and Sample Outputs	14
6.3 Performance Evaluation Metrics	15
6.4 Results and Observations	15
6.5 Limitations	16
Chapter 7: Conclusion and Future Scope	17
7.1 Conclusion	17
7.2 Future Scope	17
References	18
Appendix A: Important Code Snippets	19
Appendix B: Additional Screenshots / Outputs	20

LIST OF FIGURES

Figure Title	Page No.
Fig. 1.1: Overview of the Summarization Pipeline	3
Fig. 2.1: Existing Summarization Workflow	6
Fig. 4.1: Proposed System Architecture	8
Fig. 4.2: Workflow of the Proposed System	9
Fig. 5.1: General System Flowchart	11
Fig. 5.2: Data Flow Diagram (Level 0)	12
Fig. 5.3: Web Application User Interface	13

LIST OF TABLES

Table Title	Page No.
Table 2.1: Summary of Literature Review	5
Table 3.1: Functional Requirements	6
Table 3.2: Non-Functional Requirements	7
Table 3.3: Hardware and Software Requirements	7
Table 4.1: Dataset Description	10
Table 5.1: Tools, Technologies and Libraries Used	12
Table 6.1: Functional Test Cases	14
Table 6.2: ROUGE Score Results	15

CHAPTER 1
INTRODUCTION

1.1 Introduction to the Project:

The amount of digital text available today is enormous. Long PDF documents, Word files, reports, and articles are used for education, news, research, business, and public administration. Reading every document completely is time-consuming, and important points can be difficult to locate quickly. A summarization system helps a reader understand the key ideas of a document in a short time.

Multilingual Document Summarizer is an NLP-based application that generates abstractive summaries of documents in English and Hindi. Documents can be uploaded as .txt, .pdf, or .docx files, and users can also paste text directly.

The extracted text is sent to a Transformer-based sequence-to-sequence model. English text is summarized using BART fine-tuned on CNN/DailyMail, and Hindi text is summarized using IndicBART fine-tuned on XL-Sum. The system is exposed through a web application built with FastAPI and an HTML/CSS/JavaScript frontend, and through a Streamlit demo.

1.2 Background and Motivation

Digital technology has changed the way people read and share information. Documents are increasingly available online, yet readers often need a quick understanding of a long document rather than a complete reading, which motivates automatic summarization.

Language is another important consideration. Many readers in India are more comfortable reading Hindi than English, and most summarization tools support only English. Providing summaries in Hindi makes useful information accessible to a wider audience, and fine-tuned multilingual models such as IndicBART make this practical.

1.3 Problem Statement

To develop a multilingual NLP-based system that accepts documents, extracts their text, and generates abstractive summaries in English and Hindi through a simple web interface, so that readers can understand long content quickly without being limited by language.

1.4 Objectives

The major objectives of the project are:

1. To develop a system that summarizes long documents in English and Hindi.
2. To fine-tune Transformer-based models (BART and IndicBART) for abstractive summarization.
3. To support multiple document formats (.txt, .pdf, .docx) and pasted text.
4. To provide a clean web interface (FastAPI + HTML/CSS/JS) and a Streamlit demo.
5. To evaluate the fine-tuned models using the ROUGE metric.
6. To provide an optional Gemini API backend that runs without local models.

1.5 Scope of the Project

The project focuses on building a working bilingual summarization tool. Its scope includes:

- Uploading documents in .txt, .pdf, and .docx formats and extracting their text.
- Pasting text directly for summarization.
- Generating abstractive summaries in English and Hindi.
- Fine-tuning BART on CNN/DailyMail (English) and IndicBART on XL-Sum (Hindi).
- Evaluating summarization quality with ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-Lsum.

- A FastAPI web application, a Streamlit demo, and an optional Gemini backend.

The system is designed as an educational and practical demonstration. Summaries of very long documents are limited by the model's maximum input length.

1.6 Applications of the Project

The system can be useful in the following areas:

1. Education: Students can summarize long study material, notes, and reports.

2. News and Media: Readers can obtain quick summaries of long articles in their language.

3. Government and Administration: Citizens can summarize lengthy notices and reports.

4. Accessibility: Readers who prefer Hindi can access content in their language.

5. Research and Business: Professionals can scan reports and documents quickly.

CHAPTER 2
LITERATURE REVIEW

2.1 Review of Existing Research Papers

Natural Language Processing, abstractive summarization, and Transformer models are important research areas related to this project.

2.1.1 Attention Is All You Need

Vaswani et al. (2017) introduced the Transformer architecture, which uses self-attention to model relationships between elements of a sequence. Transformers are the foundation of modern sequence-to-sequence models such as BART and IndicBART.

2.1.2 BART: Denoising Sequence-to-Sequence Pre-training

Lewis et al. (2019) introduced BART, a denoising autoencoder that combines bidirectional and autoregressive Transformers. BART is effective for abstractive summarization and the bart-large-cnn checkpoint is a standard baseline on CNN/DailyMail.

2.1.3 IndicBART for Indian Languages

Dabre et al. (2022) introduced IndicBART, a pre-trained sequence-to-sequence model for Indian languages. IndicBART uses a compact mBART-style architecture and a language-token format that allows fine-tuning for Hindi summarization.

2.1.4 XL-Sum: Large-Scale Multilingual Abstractive Summarization

Hasan et al. (2021) introduced XL-Sum, a multilingual dataset of news article and summary pairs covering 44 languages, including Hindi. It is used in this project to fine-tune the Indic model.

Table 2.1: Summary of Literature Review

Research / resource	Main contribution	Relevance
Vaswani et al. (2017)	Transformer architecture	Foundation of BART and IndicBART
Lewis et al. (2019)	BART denoising model	English abstractive summarizer
Dabre et al. (2022)	IndicBART for Indian languages	Hindi summarization
Hasan et al. (2021)	XL-Sum multilingual dataset	Fine-tuning the Hindi model

2.2 Existing System

Existing summarization tools mostly target English text. They include cloud-based summarization APIs, browser extensions, and standalone applications that accept plain text or web pages.

Indic summarization is less commonly available. Some toolkits support Indian-language NLP for tasks such as translation and tagging, but integrated document summarization for Hindi is not readily available to general users.

2.3 Limitations of Existing System

1. Most summarization tools support English only.

2. Regional languages such as Hindi are poorly supported.

3. Uploading long documents in multiple formats is not always supported.

4. Many tools require programming knowledge or paid API access.

2.4 Research Gap

There is a need for a unified system that accepts documents in multiple formats, supports both English and Hindi, and provides summaries through a simple web interface. This project fills that gap by combining fine-tuned BART and IndicBART models with a user-friendly web front-end.

CHAPTER 3
PROBLEM STATEMENT AND REQUIREMENTS

3.1 Problem Definition

The problem addressed by the project is the difficulty readers face in quickly understanding long documents, especially when the content is in a regional language. A user requires a simple method to upload a document, choose a language, and receive a short summary without installing machine-learning libraries.

3.2 Proposed Solution

The proposed solution is a bilingual summarization application. The user uploads a document or pastes text and selects a language. The backend extracts the text and passes it to the appropriate summarization model. The generated summary is returned to the frontend.

The system is implemented using Python and Hugging Face Transformers. Document text is extracted with PyMuPDF and python-docx, and the web interface is built with FastAPI and a simple HTML/CSS/JavaScript frontend. A Streamlit version is also provided, and an optional Gemini API backend is available for lightweight deployment.

3.3 Functional Requirements

Table 3.1: Functional Requirements

ID	Requirement	Description
FR01	Upload document	Accept .txt, .pdf, and .docx files
FR02	Paste text	Accept text entered directly in the interface
FR03	Text extraction	Extract text from the uploaded document
FR04	Language selection	Allow the user to choose English or Hindi
FR05	Summarize text	Generate an abstractive summary of the extracted text
FR06	Display result	Show the summary and the extracted text
FR07	Download summary	Allow the summary to be downloaded as a text file
FR08	Model selection	Automatically use the fine-tuned model if available
FR09	Error handling	Display friendly errors for unsupported or empty input

3.4 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

Requirement	Description
Usability	The interface should be simple and easy to navigate

Performance	Summaries should be generated in a reasonable time
Reliability	Valid inputs should produce a summary without crashing
Maintainability	Code should be organized into clear modules
Scalability	Additional languages and models should be easy to add
Compatibility	The web interface should work in modern browsers
Data privacy	Uploaded documents should be processed locally

3.5 Hardware and Software Requirements

Table 3.3: Hardware Requirements

Component	Requirement
Processor	Intel Core i5 or equivalent (Apple M-series supported)
RAM	8 GB minimum; 16 GB recommended for model loading
Storage	Approximately 5 GB for models, dependencies, and caches
Display	Standard laptop or desktop display
Network	Internet required for the first model download

Table 3.4: Software Requirements

Software	Purpose
Python 3.10+	Programming language
PyTorch	Deep-learning backend
Hugging Face Transformers	BART and IndicBART models
Hugging Face Datasets	CNN/DailyMail and XL-Sum loading
PyMuPDF / python-docx	PDF and DOCX text extraction
FastAPI + Uvicorn	Web backend
Streamlit	Demo interface
HTML / CSS / JavaScript	Web frontend
Evaluate / rouge_score	ROUGE evaluation

CHAPTER 4
PROPOSED METHODOLOGY

4.1 Overview of the Proposed System

The system follows a simple client-server architecture. The frontend sends the uploaded document (or pasted text) and the selected language to the backend. The backend performs text extraction, selects the appropriate model, generates a summary, and returns it to the frontend.

1. Frontend: upload interface, language selection, and result display.
2. Text extraction: PDF, DOCX, and TXT parsing.
3. Summarization: language-specific BART or IndicBART inference (or the Gemini API).
4. Training pipeline: fine-tuning scripts and the Kaggle notebooks.

4.2 System Architecture

1. Presentation layer: HTML/CSS/JavaScript (FastAPI) and Streamlit.
2. Extraction layer: PyMuPDF and python-docx.
3. Model layer: BART (English) and IndicBART (Hindi), loaded from models/ if present.
4. Optional API layer: Google Gemini, used when SUMMARY_PROVIDER=gemini.
5. Training layer: Kaggle notebooks and scripts/train.py.

For Hindi, the input is formatted as "<text> </s> <2hi>" and the decoder is forced to start with the language token, which is required by IndicBART.

4.3 Workflow of the System

1. The user opens the web application.
2. The user uploads a document or pastes text and selects a language.
3. The backend extracts text using the appropriate extractor.
4. The text is passed to the language-specific model (or the Gemini API).
5. The model generates an abstractive summary using beam search.
6. The summary and extracted text are returned to the frontend.
7. The user can copy or download the summary.

4.4 NLP Techniques and Algorithms

4.4.1 Tokenization and Sequence-to-Sequence Modeling

The input text is tokenized and truncated to the model's maximum input length. The sequence-to-sequence model generates the summary token by token, conditioned on the encoder representation of the input.

4.4.2 Abstractive Summarization with Beam Search

Summaries are generated with beam search (num_beams = 4-5), combined with no-repeat n-gram constraints and length penalties to reduce repetition and control summary length.

4.4.3 Indic Language Handling

IndicBART uses a language-token format. The input text is appended with "</s>" and the language token, the target is prefixed with the language token, and the decoder start token is set to the language token. This ensures that the generated summary is in the correct script and language.

4.4.4 Optional Gemini Backend

When the Gemini backend is enabled, the extracted text is sent to the Gemini API with a language-aware prompt, and the returned summary is displayed. This removes the need for local model downloads.

4.5 Dataset Collection and Preprocessing

- CNN/DailyMail (English): news articles paired with human-written highlights.
- XL-Sum (Hindi): news articles paired with summaries.

XL-Sum is distributed as a script-based dataset, which newer versions of the Datasets library no longer support. The project therefore loads the auto-converted Parquet shards directly by URL.

Preprocessing includes tokenization, truncation to the model's maximum length, and, for IndicBART, language-token formatting. The English data is tokenized normally, while the Hindi data follows the "<2hi>" format described above.

Table 4.1: Dataset Description

Dataset	Language	Task	Usage
CNN/DailyMail	English	News summarization	Fine-tuning BART
XL-Sum (hindi)	Hindi	News summarization	Fine-tuning IndicBART

CHAPTER 5
SYSTEM DESIGN AND IMPLEMENTATION

5.1 System Design

The system is divided into independent modules to simplify development and maintenance.

- Input and upload interface.
- Document text extraction (PyMuPDF, python-docx).
- Language-aware summarization (BART / IndicBART).
- Optional Gemini API backend.
- Web API layer (FastAPI).
- Frontend (HTML/CSS/JavaScript) and Streamlit demo.

The modules are connected through a shared src/ package so that the web app, the Streamlit demo, and the training script use the same inference code.

5.2 Flowchart / UML / DFD Diagrams

The following diagrams describe the proposed system. They can be inserted into the final report using draw.io, Microsoft Word Shapes, or another diagram tool.

5.2.1 General System Flowchart

Upload → text extraction → language selection → model inference → summary → display/download.

Figure 5.1: General workflow of the bilingual summarization system.

5.2.2 Data Flow Diagram (Level 0)

The user provides a document and a language; the system returns a summary. The model and text-extraction components are internal processes of the application.

Figure 5.2: Level 0 Data Flow Diagram.

5.3 Tools, Technologies and Libraries

Table 5.1: Tools, Technologies and Libraries Used

Technology	Application
Python	Core programming language
PyTorch	Deep-learning backend
Hugging Face Transformers	BART and IndicBART models
Hugging Face Datasets	CNN/DailyMail and XL-Sum loading
PyMuPDF	PDF text extraction
python-docx	DOCX text extraction
FastAPI + Uvicorn	Web backend
Streamlit	Demo interface
HTML / CSS / JavaScript	Web frontend
Evaluate / rouge_score	ROUGE evaluation
Gemini API (optional)	Lightweight summarization backend

5.4 Implementation Details

5.4.1 Inference Core (src/)

src/config.py defines the model registry, provider selection, and cache handling. src/text_extraction.py extracts text from .txt, .pdf, and .docx files. src/summarizer.py loads the language-specific model and generates the summary, routing to Gemini when configured. src/llm_api.py implements the Gemini REST client.

5.4.2 Web Application

The FastAPI backend (web/server.py) exposes endpoints for text and document summarization and serves the static frontend (web/static/), which provides drag-and-drop upload, language and length controls, and copy/download actions.

5.4.3 Streamlit Demo

app.py provides the same functionality through a simple sidebar-driven Streamlit interface.

5.4.4 Training Pipeline

The models are fine-tuned using scripts/train.py and the notebooks in notebooks/. Training uses the Hugging Face Seq2SeqTrainer, and evaluation uses ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-Lsum.

5.5 User Interface and Screenshots

The web application provides a clean interface for uploading documents and for selecting the summary language. A screenshot of the running application is shown below.

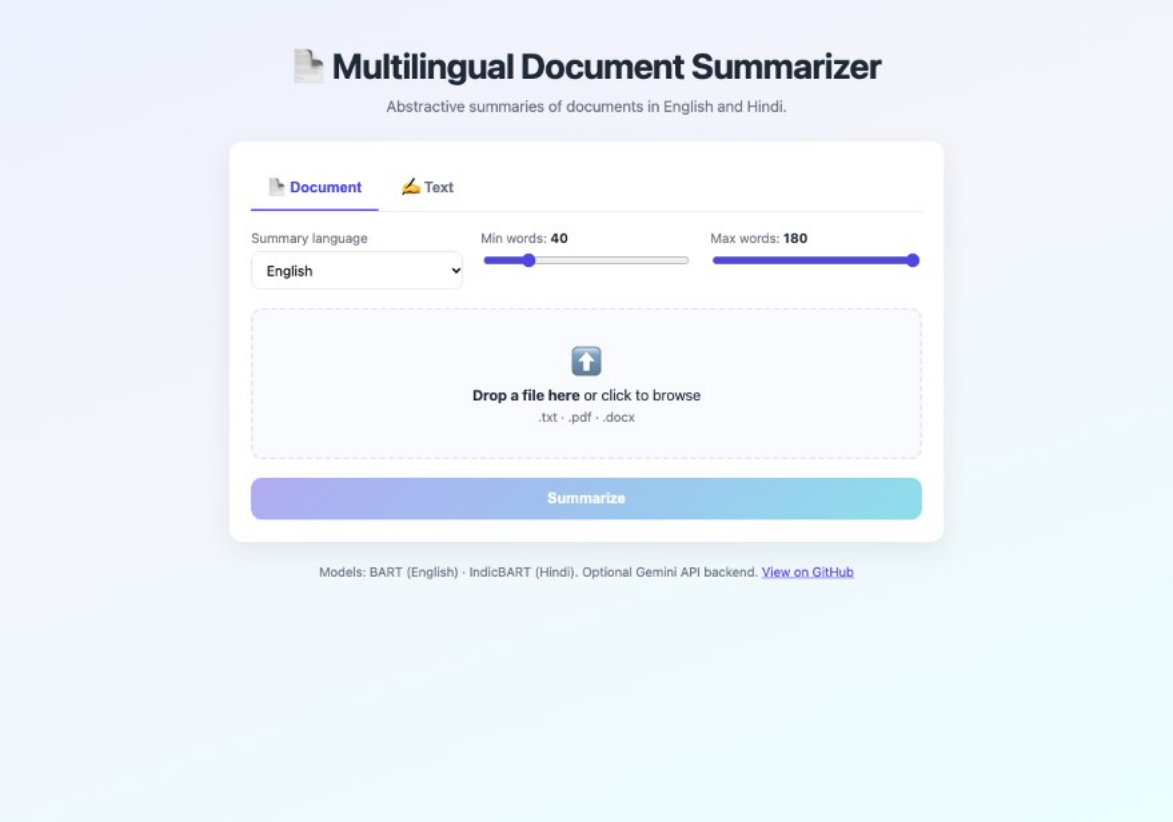


Figure 5.3: Web Application User Interface

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Experimental Setup

The models were fine-tuned on a Kaggle NVIDIA T4 GPU. BART-base was fine-tuned on CNN/DailyMail, and IndicBART was fine-tuned on the Hindi subset of XL-Sum. For each language, 2,000 training examples and 200 validation examples were used for two epochs with a batch size of 4 and a learning rate of 3e-5. Summaries were generated with beam search, and quality was measured using the ROUGE metric.

6.2 Test Cases and Sample Outputs

Table 6.1: Functional Test Cases

Test ID	Input / Action	Expected output
TC01	Open the web application	Upload interface is displayed
TC02	Upload a valid .txt file	Text is extracted and shown
TC03	Upload a valid .pdf file	Text is extracted and shown
TC04	Upload a valid .docx file	Text is extracted and shown
TC05	Paste text and summarize	Summary is generated
TC06	Select English and summarize	English summary is generated
TC07	Select Hindi and summarize	Hindi summary is generated
TC08	Upload an empty/unsupported file	A friendly error is shown
TC09	Download the summary	A .txt file is downloaded

6.3 Performance Evaluation Metrics

- ROUGE-1: unigram overlap between the generated and reference summaries.
- ROUGE-2: bigram overlap.
- ROUGE-L: longest common subsequence.
- ROUGE-Lsum: sentence-level longest common subsequence.

6.4 Results and Observations

The fine-tuned models produced the following validation results:

Table 6.2: ROUGE Score Results

Language	Model	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-Lsum
English	BART-base (CNN/DailyMail)	40.65	19.10	28.30	37.59
Hindi	IndicBART (XL-Sum)	60.42	25.90	60.36	60.26

The English model produced concise and relevant summaries. The Hindi model produced coherent summaries in the Devanagari script. Because only 2,000 examples were used for fine-tuning, the Hindi

model tends to be somewhat verbose and occasionally repeats phrases; this is expected and can be improved with more training data.

The application was verified end-to-end: documents were uploaded, text was extracted, and summaries were generated in both languages. Fine-tuned weights placed in models/ are detected automatically and used instead of the public checkpoints.

6.5 Limitations

1. The Hindi model was fine-tuned on a small sample and can be verbose.

2. Very long documents exceed the model's maximum input length and are truncated.

3. Summary quality depends on the quality of the extracted text.

4. The system runs on CPU by default; GPU improves speed.

5. The local models require a ~2 GB download on first use.

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

Multilingual Document Summarizer is an NLP-based application that makes long documents easier to understand in English and Hindi. The project integrates text extraction (PDF, DOCX, TXT), abstractive summarization using BART and IndicBART, a FastAPI web application, and a Streamlit demo.

The models were fine-tuned on Kaggle GPUs and evaluated using ROUGE. The English model achieved a ROUGE-1 of 40.65 and the Hindi model 60.42, showing that the approach is practical. The project demonstrates how modern NLP and web technologies can be combined to build a useful bilingual tool.

7.2 Future Scope

1. Fine-tune on the full datasets and more epochs for higher-quality summaries.

2. Add chunking and hierarchical summarization for very long documents.

3. Support additional Indian languages and scripts.

4. Improve the Hindi model to reduce repetition and verbosity.

5. Add voice input and text-to-speech for accessibility.

6. Deploy the application on a public server with authentication.

REFERENCES

1. Lewis, M., et al. (2019). “BART: Denoising Sequence-to-Sequence Pre-training.” arXiv:1910.13461.

2. Dabre, R., et al. (2022). “IndicBART: A Pre-trained Model for Natural Language Generation of Indic Languages.” arXiv:2109.02903.

3. Hasan, T., et al. (2021). “XL-Sum: Large-Scale Multilingual Abstractive Summarization for 44 Languages.” Findings of ACL-IJCNLP 2021.

4. Vaswani, A., et al. (2017). “Attention Is All You Need.” Advances in Neural Information Processing Systems.

5. Hermann, K. M., et al. (2015). “Teaching Machines to Read and Comprehend.” NeurIPS. (CNN/DailyMail)

6. Hugging Face Transformers. https://huggingface.co/docs/transformers/

7. PyMuPDF Documentation. https://pymupdf.readthedocs.io/

8. FastAPI Documentation. https://fastapi.tiangolo.com/

9. Streamlit Documentation. https://docs.streamlit.io/

10. Google Gemini API. https://ai.google.dev/

APPENDIX A: IMPORTANT CODE SNIPPETS

Short, relevant excerpts from the project source code are given below.

A.1 Summarization Function

Source file: src/summarizer.py

The summarize() function loads the language-specific model, tokenizes the input, applies the IndicBART language format where required, and generates the summary with beam search. It routes to the Gemini API when SUMMARY_PROVIDER=gemini.

```
def summarize(text, language, max_length=None, min_length=None):
    if SUMMARY_PROVIDER == "gemini":
        return summarize_with_gemini(text, language, max_length, min_length)
    tokenizer, model = _load_pipeline(language)
    inputs = tokenizer(text, max_length=512, truncation=True, return_tensors="pt")
    output = model.generate(**inputs, num_beams=4, max_length=128)
    return _decode_clean(tokenizer, output[0].tolist())
```

A.2 Text Extraction

Source file: src/text_extraction.py

The extract_text() function dispatches to the correct reader based on the file extension.

```
def extract_text(path):
    suffix = Path(path).suffix.lower()
    if suffix == ".txt":  return extract_text_from_txt(path)
    if suffix == ".pdf":  return extract_text_from_pdf(path)
    if suffix == ".docx": return extract_text_from_docx(path)
    raise ValueError(f"Unsupported document type: {suffix}")
```

A.3 FastAPI Document Endpoint

Source file: web/server.py

```
@app.post("/api/summarize-document")
async def summarize_document(file: UploadFile, language: str = Form("en")):
    text = extract_text(file)
    summary = summarize(text, language)
    return {"summary": summary, "source_text": text}
```

APPENDIX B: ADDITIONAL SCREENSHOTS / OUTPUTS

Sample outputs generated by the system are shown below.

Table B.1: Sample Outputs

Language	Input (excerpt)	Generated summary (excerpt)
English	Artificial intelligence is transforming many industries…	Machine learning models can now read documents and images.
Hindi	भारत सरकार ने डिजिटल शिक्षा को बढ़ावा देने के लिए योजना…	भारत सरकार ने नई योजना की घोषणा की है।